LAPORAN TEKNIS

Instalasi dan Konfigurasi Application Server, Deployment System, serta Analisis Kebutuhan Akses Internet

Sistem CMS Website — PT Terminal Petikemas Surabaya

Nomor Dokumen	TPS-PRIMALINK/APP/2026-002
Versi	1.0
Tanggal	12 Februari 2026
Klasifikasi	Rahasia / Internal
Status	Final

DIPERSIAPKAN UNTUK

PT Terminal Petikemas Surabaya

PT PRIMALINK

Technology Solutions Partner

LEMBAR PENGESAHAN

Judul Dokumen	Laporan Instalasi Application Server, Deployment System, serta Analisis Kebutuhan Akses Internet untuk Sistem CMS Website PT Terminal Petikemas Surabaya
Nomor Dokumen	TPS-PRIMALINK/APP/2026-002
Versi	1.0

Disiapkan Oleh	Diperiksa Oleh	Disetujui Oleh
Tim Teknis PT PRIMALINK	**Project Manager** PT PRIMALINK	**PIC IT** PT Terminal Petikemas Surabaya
Tanggal: ___/___/2026	Tanggal: ___/___/2026	Tanggal: ___/___/2026

Riwayat Perubahan Dokumen

Versi	Tanggal	Perubahan	Penulis
0.1	03 Feb 2026	Draft awal — instalasi Docker dan deployment aplikasi	Tim Teknis
0.2	12 Feb 2026	Bug fixing upload file, konfigurasi routing, Prisma init	Tim Teknis
0.3	12 Feb 2026	Setup volume mount deployment, analisis kebutuhan internet	Tim Teknis
1.0	12 Feb 2026	Finalisasi dokumen	Tim Teknis

DAFTAR ISI

BAB I — PENDAHULUAN

1.1 Latar Belakang

1.2 Tujuan Dokumen

1.3 Ruang Lingkup

BAB II — SPESIFIKASI APPLICATION SERVER

2.1 Spesifikasi Hardware dan Software

2.2 Arsitektur Aplikasi

2.3 Teknologi yang Digunakan

BAB III — INSTALASI DAN KONFIGURASI

3.1 Login ke Application Server

3.2 Instalasi Docker Engine

3.3 Persiapan Direktori Aplikasi

3.4 Membuat File Dockerfile

3.5 Membuat File Docker Compose

3.6 Konfigurasi Volume Mount

3.7 Transfer Source Code ke Server

3.8 Migrasi File Upload

3.9 Build dan Jalankan Container

3.10 Verifikasi Deployment

BAB IV — KENDALA DAN PENYELESAIAN

4.1 Prisma Client Initialization Failure

4.2 File Upload Tidak Berfungsi

4.3 Routing File URL Tidak Konsisten

4.4 Encoding Spasi pada URL File Legacy

BAB V — SISTEM DEPLOYMENT

5.1 Workflow Deployment Saat Ini

5.2 Deploy Script (deploy.sh)

5.3 Keterbatasan Tanpa Akses Internet

BAB VI — ANALISIS KEBUTUHAN AKSES INTERNET

6.1 Kondisi Saat Ini

6.2 Dampak Ketiadaan Akses Internet

6.3 Kebutuhan Internet untuk CI/CD Pipeline

6.4 Kebutuhan Internet untuk Package Management

6.5 Kebutuhan Internet untuk Security Updates

6.6 Kebutuhan Internet untuk Docker Registry

6.7 Kebutuhan Internet untuk Monitoring dan Alerting

6.8 Perbandingan Workflow: Tanpa Internet vs Dengan Internet

6.9 Rekomendasi Konfigurasi Akses Internet

BAB VII — PENGUJIAN DAN VALIDASI

7.1 Pengujian Fungsional

7.2 Pengujian Deployment

BAB VIII — KESIMPULAN DAN REKOMENDASI

8.1 Kesimpulan

8.2 Rekomendasi

LAMPIRAN

BAB I
PENDAHULUAN

1.1 Latar Belakang

Sebagai bagian dari proyek pengembangan sistem CMS (Content Management System) website PT Terminal Petikemas Surabaya, telah dilakukan instalasi dan konfigurasi application server pada infrastruktur jaringan internal TPS. Application server berfungsi sebagai host dari aplikasi CMS yang melayani website publik dan panel admin untuk pengelolaan konten.

Dalam proses implementasi, ditemukan beberapa kendala teknis yang memerlukan penanganan khusus, terutama terkait dengan keterbatasan akses internet pada server. Kendala ini berdampak signifikan terhadap proses deployment, pemeliharaan, dan pengembangan berkelanjutan dari sistem CMS.

Dokumen ini memaparkan secara lengkap proses instalasi, kendala yang dihadapi beserta solusinya, serta analisis mendalam mengenai mengapa akses internet sangat diperlukan untuk mengakomodir proses deployment dan pemeliharaan server secara optimal.

1.2 Tujuan Dokumen

Dokumen ini disusun dengan tujuan:

1. Mendokumentasikan proses instalasi dan konfigurasi application server (172.19.154.93).
2. Mendokumentasikan kendala teknis yang ditemui selama proses implementasi beserta solusinya.
3. Menjelaskan sistem deployment yang saat ini diterapkan dan keterbatasannya.

4. **Memberikan analisis dan justifikasi teknis mengapa akses internet sangat dibutuhkan** untuk menunjang proses deployment, pemeliharaan, dan pengembangan server.
5. Memberikan rekomendasi konfigurasi akses internet yang aman.

1.3 Ruang Lingkup

Ruang lingkup dokumen ini meliputi:

1. Instalasi Docker Engine dan konfigurasi container aplikasi pada server 172.19.154.93.
2. Konfigurasi volume mount untuk deployment system.
3. Dokumentasi seluruh kendala teknis dan bug fixing yang dilakukan.
4. Analisis komprehensif kebutuhan akses internet untuk server.

BAB II

SPESIFIKASI APPLICATION SERVER

2.1 Spesifikasi Hardware dan Software

Parameter	Nilai
Hostname	`tpswebsiteapp`
IP Address	172.19.154.93
Sistem Operasi	Ubuntu 24.04.3 LTS (Noble Numbat)
Kernel	Linux 6.8.0-90-generic x86_64
CPU	4 Core
RAM	8 GB
Disk	200 GB (LVM), terpakai 24 GB (12%)
Docker Engine	Docker 29.2.1
Uptime	12+ hari (stabil)

Tabel 2.1 — Spesifikasi Application Server

2.2 Arsitektur Aplikasi

```
┌──────────────────────────────────────────────────────────────┐
│              APPLICATION SERVER (172.19.154.93)              │
│                                                              │
│  ┌──────────────── Docker Container ────────────────────┐    │
│  │  Container: tps-app                                  │    │
│  │  Image: oven/bun:1.1.20-debian (1.17 GB)             │    │
│  │                                                      │    │
│  │  ┌──────────────────────────────────────────────┐   │    │
│  │  │  Bun.js Runtime v1.1.20                      │   │    │
│  │  │  ┌──────────────┐  ┌──────────────────────┐  │   │    │
│  │  │  │ Prasi Bundle │  │ SSR API Endpoints    │  │   │    │
│  │  │  │ (Legacy CMS) │  │ (Admin Backend)      │  │   │    │
│  │  │  └──────────────┘  └──────────────────────┘  │   │    │
│  │  │  ┌──────────────┐  ┌──────────────────────┐  │   │    │
│  │  │  │ Prisma ORM   │  │ File Upload Handler  │  │   │    │
│  │  │  │ (DB Client)  │  │ (/_upload, /_file)   │  │   │    │
│  │  │  └──────────────┘  └──────────────────────┘  │   │    │
│  │  └──────────┬───────────────────────────────────┘   │    │
│  │             │ TCP:5432                                │    │
│  └─────────────┼─────────────────────────────────────────┘    │
│                │        Port 3300 (HTTP) ↕ User/Admin         │
│                ▼                                              │
│      ┌──────────────────┐                                     │
│      │  172.19.154.92   │                                     │
│      │  PostgreSQL 16   │                                     │
│      │  Database: tps_web │                                   │
│      └──────────────────┘                                     │
└──────────────────────────────────────────────────────────────┘
```

Gambar 2.1 — Arsitektur Application Server dan Koneksi Database

2.3 Teknologi yang Digunakan

Komponen	Teknologi	Versi	Keterangan
Container Runtime	Docker	29.2.1	Isolasi aplikasi
JavaScript Runtime	Bun	1.1.20	Server-side JavaScript (pengganti Node.js)
Database ORM	Prisma	5.11.0	Type-safe database access
CMS Engine	Prasi	Custom	Content management & page rendering
Process Manager	PM2-style (built-in)	-	Auto-restart on crash
Dependencies	247 npm packages	Various	Termasuk firebase-admin, brotli-wasm, dll

Tabel 2.2 — Stack Teknologi

BAB III
INSTALASI DAN KONFIGURASI

PRASYARAT: Sebelum memulai instalasi Application Server, pastikan **PostgreSQL Database Server sudah terinstall dan berjalan** di server 172.19.154.92. Lihat dokumen *TPS-PRIMALINK/DB/2026-001 (Laporan Instalasi PostgreSQL)* untuk panduan instalasi database.

3.1 Login ke Application Server

Langkah pertama adalah login ke application server (172.19.154.93) melalui SSH. Buka aplikasi terminal (misalnya: PuTTY di Windows, atau Terminal di Mac/Linux).

1 Login ke Server via SSH

Ketik perintah berikut di terminal, lalu tekan **Enter**:

```
ssh root@172.19.154.93
```

Jika diminta password, masukkan password server lalu tekan **Enter**. (Password tidak akan terlihat saat diketik — ini normal.)

Jika muncul pertanyaan `Are you sure you want to continue connecting (yes/no)?`, ketik **yes** lalu tekan **Enter**.

Anda berhasil login jika tampilan berubah menjadi: `root@tpswebsiteapp:~#`

3.2 Instalasi Docker Engine

Docker Engine diinstall pada server untuk menjalankan aplikasi CMS dalam lingkungan container yang terisolasi. Container seperti "kotak" yang berisi aplikasi beserta semua kebutuhannya, sehingga aplikasi berjalan konsisten di mana saja.

2 Download Script Instalasi Docker

Ketik perintah berikut lalu tekan **Enter**:

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

Perintah ini mendownload script instalasi Docker resmi dari internet. Tidak ada output jika berhasil.

3 Jalankan Script Instalasi Docker

Ketik perintah berikut lalu tekan **Enter**. **Tunggu hingga proses selesai** (bisa memakan waktu 2-5 menit):

```
sudo sh get-docker.sh
```

Anda akan melihat banyak output berupa proses download dan instalasi. Proses selesai jika muncul kembali tampilan `root@tpswebsiteapp:~#`.

4 Verifikasi Instalasi Docker

Ketik perintah berikut lalu tekan **Enter**:

```
docker --version
```

Harus muncul output seperti ini (versi bisa sedikit berbeda):

```
Docker version 29.2.1, build a5c7197
```

5 **Aktifkan Docker Autostart**

Ketik dua perintah berikut satu per satu, tekan **Enter** setelah masing-masing:

```
sudo systemctl enable docker
sudo systemctl start docker
```

Perintah pertama memastikan Docker otomatis berjalan ketika server di-restart. Perintah kedua menjalankan Docker sekarang.

3.3 Persiapan Direktori Aplikasi

Sebelum membuat container, kita perlu menyiapkan direktori-direktori yang akan digunakan oleh aplikasi.

6 **Buat Direktori Aplikasi dan Data**

Ketik perintah berikut lalu tekan **Enter**:

```
mkdir -p /opt/tps-app /opt/tps-data/uploads /opt/tps-data/logs
```

Penjelasan direktori:

- `/opt/tps-app` — Tempat source code aplikasi
- `/opt/tps-data/uploads` — Tempat file upload (gambar, dokumen, dll)
- `/opt/tps-data/logs` — Tempat log aplikasi

7 **Pindah ke Direktori Aplikasi**

Ketik perintah berikut lalu tekan **Enter**:

```
cd /opt/tps-app
```

Tampilan akan berubah menjadi `root@tpswebsiteapp:/opt/tps-app#`

3.4 Membuat File Dockerfile

Dockerfile adalah "resep" untuk membuat Docker image — berisi instruksi langkah demi langkah untuk menyiapkan lingkungan aplikasi.

8 **Buat File Dockerfile**

Ketik perintah berikut lalu tekan **Enter** untuk membuka editor:

```
nano /opt/tps-app/Dockerfile
```

File akan terbuka (kosong). **Salin dan tempelkan (paste)** seluruh isi berikut ke dalam editor:

```
FROM oven/bun:1.1.20-debian

RUN apt-get update && apt-get install -y curl git openssl \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY . ./
RUN bun install --production
RUN mkdir -p /app/uploads /app/logs
RUN cd app/db && rm -f .env && \
    bunx prisma generate --schema=prisma/schema.prisma

ENV NODE_ENV=production
ENV PORT=3300
EXPOSE 3300

HEALTHCHECK --interval=30s --timeout=10s --start-period=40s --
retries=3 \
    CMD curl -f http://localhost:3300/ || exit 1

CMD ["bun", "run", "prod"]
```

Setelah selesai paste:

1. Tekan **Ctrl+O** untuk menyimpan, lalu tekan **Enter**.
2. Tekan **Ctrl+X** untuk keluar dari editor.

Parameter	Nilai
Image Name	`tps-app-tps-app:latest`
Image Size	1.17 GB
Base Image	`oven/bun:1.1.20-debian`
Total Build Cache	3.48 GB

Tabel 3.1 — Informasi Docker Image

3.5 Membuat File Docker Compose

Docker Compose adalah file konfigurasi yang mendefinisikan bagaimana container dijalankan — termasuk port, environment variables, volume mounts, dan health check.

9 Buat File docker-compose.yml

Ketik perintah berikut lalu tekan **Enter** untuk membuka editor:

```
nano /opt/tps-app/docker-compose.yml
```

File akan terbuka (kosong). **Salin dan tempelkan (paste)** seluruh isi berikut ke dalam editor:

```
services:
  tps-app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: tps-app
    restart: unless-stopped
    ports:
      - "3300:3300"
    environment:
      - NODE_ENV=production
      - DATABASE_URL=postgresql://
postgres:TpsDB2026Secure@172.19.154.92:5432/tps_web
      - PORT=3300
    volumes:
      # Source code mounts (synced via deploy.sh)
      - /opt/tps-app/pkgs:/app/pkgs
      - /opt/tps-app/app/srv:/app/app/srv
      - /opt/tps-app/app/db:/app/app/db
      - /opt/tps-app/app/web/deploy:/app/app/web/deploy
      # Data mounts
      - /opt/tps-data/uploads:/data/files
      - /opt/tps-data/logs:/app/logs
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3300/"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
```

Setelah selesai paste:

1. Tekan **Ctrl+O** untuk menyimpan, lalu tekan **Enter**.
2. Tekan **Ctrl+X** untuk keluar dari editor.

PENTING: Pada baris `DATABASE_URL`, pastikan password (`TpsDB2026Secure`) dan IP database (`172.19.154.92`) sesuai dengan konfigurasi yang dibuat saat instalasi PostgreSQL. Jika password berbeda, ganti sesuai password yang Anda set.

Tips Editor Nano:

- **Ctrl+O** = Simpan file (lalu tekan Enter)
- **Ctrl+X** = Keluar dari editor
- **Ctrl+W** = Cari teks
- **Panah** ↑ ↓ ← → = Navigasi kursor
- Di terminal, **paste** biasanya menggunakan **Ctrl+Shift+V** (Linux) atau **klik kanan** (PuTTY di Windows)

3.6 Konfigurasi Volume Mount

Volume mount menghubungkan direktori di server host dengan direktori di dalam container. Dengan cara ini, perubahan kode di server langsung terlihat oleh aplikasi di dalam container tanpa perlu rebuild image.

Host Path	Container Path	Fungsi
`/opt/tps-app/pkgs`	`/app/pkgs`	Server framework (routing, upload handler, utils)
`/opt/tps-app/app/srv`	`/app/app/srv`	SSR API endpoints & admin components
`/opt/tps-app/app/db`	`/app/app/db`	Prisma schema & database config
`/opt/tps-app/app/web/deploy`	`/app/app/web/deploy`	Prasi bundled files (.gz)
`/opt/tps-data/uploads`	`/data/files`	File uploads (4.7 GB)
`/opt/tps-data/logs`	`/app/logs`	Application logs

Tabel 3.2 — Daftar Volume Mount

> **Catatan:** `node_modules` (247 packages) tetap berada di dalam Docker image dan **tidak** di-mount dari host. Hal ini memastikan dependencies selalu konsisten dan tidak perlu install Bun/npm pada host server.

3.7 Transfer Source Code ke Server

Source code aplikasi perlu ditransfer dari komputer developer ke server. Proses ini dilakukan menggunakan script `deploy.sh` atau secara manual menggunakan `rsync` / `scp`.

10 **Transfer Source Code (dari komputer developer)**

Perintah ini dijalankan **dari komputer developer (bukan dari server)**. Buka terminal baru di komputer Anda, pindah ke folder project, lalu jalankan:

```
rsync -az tps-core/tps/pkgs/ root@172.19.154.93:/opt/tps-app/
pkgs/
rsync -az tps-core/tps/app/srv/ root@172.19.154.93:/opt/tps-app/
app/srv/
rsync -az tps-core/tps/app/db/ root@172.19.154.93:/opt/tps-app/
app/db/
rsync -az tps-core/tps/app/web/deploy/ root@172.19.154.93:/opt/
tps-app/app/web/deploy/
```

Ketik setiap baris satu per satu, tekan **Enter** setelah masing-masing. Jika diminta password, masukkan password server. Tunggu setiap transfer selesai sebelum mengetik perintah berikutnya.

Atau, jika script deploy.sh sudah tersedia, cukup jalankan:

```
./deploy.sh --sync-only
```

3.8 Migrasi File Upload

File upload dari production existing (total 4.7 GB, berisi gambar, dokumen, dll) perlu dimigrasikan ke server baru.

11 **Transfer File Upload ke Server (dari komputer developer)**

Perintah ini dijalankan **dari komputer developer**:

```
scp files_backup.tar.gz root@172.19.154.93:/opt/tps-data/
```

File `files_backup.tar.gz` harus ada di folder Anda saat ini. Transfer bisa memakan waktu lama (tergantung kecepatan jaringan dan ukuran file).

12 **Extract File Upload di Server**

Kembali ke terminal server (172.19.154.93). Ketik perintah berikut satu per satu:

```
cd /opt/tps-data
tar xzf files_backup.tar.gz -C uploads/
```

Penjelasan:

- `cd /opt/tps-data` = pindah ke direktori data
- `tar xzf` = extract file .tar.gz
- `-C uploads/` = extract ke dalam folder uploads

Proses extract bisa memakan waktu beberapa menit. Tunggu sampai kembali ke tampilan `root@tpswebsiteapp:~#`

13 **Verifikasi File Upload**

Ketik perintah berikut lalu tekan **Enter**:

```
du -sh /opt/tps-data/uploads/
```

Harus muncul ukuran sekitar **4.7G** (atau sesuai ukuran file upload asli).

3.9 Build dan Jalankan Container

Setelah semua file siap, kita dapat membuat Docker image dan menjalankan container aplikasi.

14 **Build Docker Image**

Pastikan Anda berada di direktori `/opt/tps-app`. Ketik perintah berikut lalu tekan **Enter**:

```
cd /opt/tps-app
docker compose build
```

Proses ini akan memakan waktu 5-15 menit tergantung kecepatan server dan koneksi internet (untuk download base image dan dependencies). Anda akan melihat banyak output berupa proses download dan instalasi.

Proses selesai jika muncul pesan seperti: `Successfully built ...` atau `Successfully tagged ...`

15 **Jalankan Container**

Ketik perintah berikut lalu tekan **Enter**:

```
docker compose up -d
```

Parameter `-d` artinya container berjalan di background (tidak menampilkan log terus-menerus). Jika berhasil, akan muncul pesan: `Container tps-app Started`

16 **Tunggu Container Siap (Health Check)**

Tunggu sekitar **40 detik** agar container selesai startup, lalu cek status:

```
docker ps
```

Harus muncul output yang menunjukkan container `tps-app` dengan status **Up ... (healthy)**:

```
CONTAINER ID   IMAGE           STATUS                    PORTS
abcd1234...    tps-app-tps-app Up 45 seconds (healthy)
0.0.0.0:3300->3300/tcp
```

Jika status masih `(health: starting)`, tunggu 30 detik lagi lalu jalankan `docker ps` lagi.

Jika status `(unhealthy)`, lihat log error dengan perintah: `docker logs tps-app`

3.10 Verifikasi Deployment

Setelah container berjalan, lakukan verifikasi untuk memastikan semua komponen berfungsi dengan baik.

17 **Test Akses Homepage**

Ketik perintah berikut lalu tekan **Enter**:

```
curl -s -o /dev/null -w "%{http_code}" http://localhost:3300/
```

Harus muncul angka: **200** (artinya homepage berhasil diakses).

Anda juga bisa mengakses dari browser komputer lain di jaringan yang sama dengan membuka: `http://172.19.154.93:3300/`

18 Cek Volume Mount

Ketik perintah berikut lalu tekan **Enter**:

```
docker inspect tps-app --format '{{range .Mounts}}{{.Source}} ->
{{.Destination}}{{"\n"}}{{end}}'
```

Harus muncul 6 baris mount yang menunjukkan koneksi antara direktori host dan container.

19 Cek Log Aplikasi (Opsional)

Untuk melihat log aplikasi secara real-time, ketik:

```
docker logs -f tps-app
```

Log akan terus berjalan. Tekan **Ctrl+C** untuk berhenti melihat log (container tetap berjalan).

Item	Expected	Actual	Status
Container running	Up (healthy)	Up (healthy)	✅ Pass
Port 3300 listening	LISTEN	LISTEN	✅ Pass
Homepage accessible	HTTP 200	HTTP 200 (237 KB)	✅ Pass
DB connection	Connected	17.773 records	✅ Pass
File serving	Files accessible	4.7 GB files served	✅ Pass
Admin panel	HTTP 200	HTTP 200	✅ Pass
6 volume mounts	All mounted	All mounted (bind)	✅ Pass

Tabel 3.3 — Checklist Verifikasi Deployment

BAB IV
KENDALA DAN PENYELESAIAN

Selama proses implementasi dan pengoperasian application server, ditemukan beberapa kendala teknis yang memerlukan investigasi dan perbaikan. Berikut dokumentasi lengkap setiap kendala beserta solusinya.

4.1 Prisma Client Initialization Failure

Severity	**CRITICAL** — Seluruh halaman website tidak dapat diakses
Tanggal Ditemukan	12 Februari 2026
File Terkait	`pkgs/utils/prisma.ts`

Deskripsi Masalah

Seluruh halaman website (homepage, berita, unduh dokumen, dll) menampilkan error atau halaman kosong. Log container menunjukkan error berulang: `TypeError: undefined is not an object (evaluating 'db.config')`.

Root Cause Analysis

Prisma ORM versi 5.11.0 memiliki bug pada schema parser (`psl/parser-database/src/walkers/relation/inline.rs`) yang menyebabkan perintah `prisma generate` gagal dengan error *"index out of bounds: the len is 0 but the index is 0"*.

Dalam kode original, perintah `prisma generate` dan `import PrismaClient` berada dalam satu blok `try-catch` yang sama. Ketika `prisma generate` gagal, seluruh blok di-skip — termasuk `import PrismaClient` yang sebenarnya masih bisa berfungsi menggunakan client yang sudah di-generate sebelumnya saat Docker build.

```
# Kode sebelum perbaikan (BERMASALAH)
try {
  await prisma generate;      // ← GAGAL (bug Prisma 5.11.0)
  g.db = new PrismaClient();  // ← TIDAK PERNAH DIEKSEKUSI
} catch (e) {
  // g.db tetap undefined → seluruh website error
}
```

Solusi

Memisahkan `prisma generate` dan `import PrismaClient` ke dalam dua blok `try-catch` independen, sehingga kegagalan generate tidak menghalangi penggunaan client yang sudah ada.

```
# Kode setelah perbaikan
try {
  await prisma generate;
} catch (e) {
  console.log("Using existing client");  // Non-fatal
}

try {
  g.db = new PrismaClient();  // ← Tetap dieksekusi
} catch (e) {
  console.log("Prisma not initialized");
}
```

4.2 File Upload Tidak Berfungsi

Severity	**HIGH** — Admin tidak dapat upload dokumen/media
Tanggal Ditemukan	12 Februari 2026
File Terkait	`app/srv/api/_tpsadmin_edit.ts`, `pkgs/api/_upload.ts`

Deskripsi Masalah

Fitur upload file pada halaman admin edit content (`/backend/tpsadmin/edit/:id`) tidak berfungsi. Terdapat tiga bug yang saling terkait:

Bug 1: Request Body Format Salah

Frontend mengirim raw `File` object sebagai request body, padahal backend menggunakan multipart parser yang membutuhkan `FormData` dengan boundary.

```
# Sebelum (SALAH)
fetch('/_upload', { method: 'POST', body: file })

# Sesudah (BENAR)
const formData = new FormData();
formData.append('file', file);
fetch('/_upload', { method: 'POST', body: formData })
```

Bug 2: Direktori Upload Tidak Dibuat

Backend menggunakan path relatif untuk mengecek dan membuat direktori upload, serta memiliki logika yang terbalik (hanya membuat direktori jika sudah ada).

```
# Sebelum (SALAH — relative path + logika terbalik)
if (await existsAsync(dirname(to))) {
  dirAsync(dirname(to));  // No-op: sudah ada
}

# Sesudah (BENAR — absolute path, selalu buat)
const fullPath = dir(`${g.datadir}/files/${to}`);
await dirAsync(dirname(fullPath));
```

Bug 3: Page Reload Sebelum Save

Setelah upload berhasil, halaman langsung melakukan `location.reload()` tanpa menyimpan file path ke database terlebih dahulu, sehingga path yang baru diupload hilang.

Solusi: Mengganti `location.reload()` dengan update preview langsung di DOM, sehingga user bisa melihat file yang diupload dan menyimpannya melalui tombol "Simpan".

4.3 Routing File URL Tidak Konsisten

Severity	**MEDIUM** — File download dari halaman unduh dokumen error 404
File Terkait	`pkgs/server/create.ts`

Deskripsi Masalah

Halaman Prasi legacy (unduh-dokumen) menghasilkan URL file dengan pola `/file/...`, sedangkan endpoint server hanya menangani `/_file/...` (dengan underscore). Akibatnya, klik download file menghasilkan error 404 Not Found.

Solusi

Menambahkan redirect permanen (HTTP 301) dari `/file/*` ke `/_file/*` pada server routing.

4.4 Encoding Spasi pada URL File Legacy

Severity	**MEDIUM** — File dengan spasi di nama tidak bisa didownload
File Terkait	`pkgs/server/create.ts`

Deskripsi Masalah

Prasi legacy mengencode karakter spasi sebagai tilde (`~`) pada URL file, sedangkan file di disk menggunakan spasi. Contoh: URL `/file/media/laporan~penggantian~ssl.pdf` mengarah ke file `laporan penggantian ssl.pdf`.

Solusi

Menambahkan konversi `~` ke `%20` (spasi) pada redirect handler.

BAB V

SISTEM DEPLOYMENT

5.1 Workflow Deployment Saat Ini

Karena server tidak memiliki akses internet, proses deployment tidak dapat menggunakan mekanisme standar seperti `git pull` atau CI/CD pipeline. Saat ini, deployment dilakukan melalui sinkronisasi file manual dari komputer developer ke server menggunakan SSH dan `rsync`.

```
┌──────────────────────────────────────────────────────────────────┐
│                   WORKFLOW DEPLOYMENT SAAT INI                   │
│                                                                  │
│  ┌──────────────┐       ┌──────────────┐       ┌────────────────┐ │
│  │  Developer   │ SSH   │  Server      │       │   Docker       │ │
│  │  (Local PC)  │──────▶│  Host        │──────▶│   Container    │ │
│  │              │rsync  │ /opt/tps-    │volume │   tps-app      │ │
│  │  ./deploy.sh │       │  app/        │mount  │   /app/        │ │
│  └──────────────┘       └──────────────┘       └────────────────┘ │
│       │                                                          │
│       │ git push                                                 │
│       ▼                                                          │
│  ┌──────────────┐                                                │
│  │   GitHub     │   ← TIDAK BISA DIAKSES DARI SERVER             │
│  │  (Backup)    │       (Server tanpa internet)                  │
│  └──────────────┘                                                │
└──────────────────────────────────────────────────────────────────┘
```

Gambar 5.1 — Workflow Deployment Saat Ini (Tanpa Internet)

5.2 Deploy Script (deploy.sh)

Script deployment otomatis telah dibuat untuk menyederhanakan proses. Script ini melakukan sinkronisasi 4 direktori source code, restart container, dan verifikasi health check.

> **Catatan:** Script ini dijalankan dari **komputer developer** (bukan dari server). Pastikan Anda berada di folder project `tps-core/tps/` sebelum menjalankan script.

Cara menjalankan deploy script:

Langkah 1: Buka Terminal di komputer developer Anda, lalu masuk ke folder project:

```
cd tps-core/tps
```

Langkah 2: Jalankan script deploy sesuai kebutuhan:

```
# Deploy lengkap (sync code + restart container):
./deploy.sh

# Hanya sync code tanpa restart (jika hanya update konten):
./deploy.sh --sync-only

# Hanya restart container (jika tidak ada perubahan code):
./deploy.sh --restart
```

Langkah 3: Tunggu proses selesai. Script akan menampilkan status untuk setiap tahap:

```
# Contoh output yang normal:
sending incremental file list   ← rsync sedang mengirim file
...
tps-app                         ← container berhasil di-restart
HTTP 200                        ← website bisa diakses (berhasil!)
```

> **Isi script deploy.sh (untuk referensi):**

```
#!/bin/bash
# Usage:
#   ./deploy.sh              # Sync + restart
#   ./deploy.sh --sync-only  # Sync + clear cache
#   ./deploy.sh --restart    # Restart only

# 1. Sync source code via rsync (SSH)
rsync -az pkgs/     root@172.19.154.93:/opt/tps-app/pkgs/
rsync -az app/srv/  root@172.19.154.93:/opt/tps-app/app/srv/
rsync -az app/db/   root@172.19.154.93:/opt/tps-app/app/db/
rsync -az app/web/deploy/ root@172.19.154.93:/opt/tps-app/app/web/
deploy/

# 2. Restart container
ssh root@172.19.154.93 "docker restart tps-app"

# 3. Health check & smoke test
# → Verifikasi container Up (healthy) dan homepage HTTP 200
```

Jika deploy gagal:

• **"Permission denied"** → Pastikan SSH key sudah terdaftar di server (lihat BAB III bagian 3.1)

• **"Connection refused"** → Pastikan server menyala dan bisa diakses via jaringan

• **"rsync: command not found"** → Install rsync: `sudo apt install rsync` (di komputer developer)

5.3 Keterbatasan Tanpa Akses Internet

KETERBATASAN KRITIS: Server 172.19.154.93 **tidak memiliki akses internet sama sekali**. Hasil pengujian menunjukkan bahwa koneksi ke GitHub.com dan Google.com timeout (HTTP 000). Hal ini menyebabkan beberapa keterbatasan serius yang dijabarkan pada BAB VI.

```
# Bukti tidak ada akses internet
$ curl --connect-timeout 5 https://github.com
# → TIMEOUT (HTTP 000)

$ curl --connect-timeout 5 https://google.com
# → TIMEOUT (HTTP 000)
```

BAB VI
ANALISIS KEBUTUHAN AKSES INTERNET

6.1 Kondisi Saat Ini

Berdasarkan pengujian yang dilakukan pada 12 Februari 2026, server application (172.19.154.93) tidak memiliki akses internet. Seluruh upaya koneksi ke layanan eksternal (GitHub, Google, npm registry, Docker Hub) menghasilkan timeout. Kondisi ini menyebabkan sejumlah dampak operasional yang signifikan.

6.2 Dampak Ketiadaan Akses Internet

Berikut adalah dampak langsung yang sudah terjadi akibat tidak adanya akses internet pada server:

No	Dampak	Severity	Penjelasan
1	Deployment manual & lambat	**HIGH**	Setiap perubahan kode harus di-rsync manual dari PC developer. Proses ini bergantung pada satu orang yang memiliki akses SSH, menciptakan single point of failure.
2	Tidak bisa update dependencies	**HIGH**	Server tidak bisa menjalankan `bun install` atau `npm install` untuk update/patch library. Jika ditemukan vulnerability pada library, patching tidak bisa dilakukan langsung.
3	Tidak bisa rebuild Docker image	**HIGH**	Proses `docker build` membutuhkan download base image dan dependencies dari internet. Tanpa internet, Docker image harus di-build di tempat lain lalu di-transfer manual.
4	Tidak ada automated deployment	**MEDIUM**	CI/CD pipeline (GitHub Actions, GitLab CI) tidak bisa diterapkan karena server tidak bisa pull dari repository.
5	Tidak ada security updates	**HIGH**	Server tidak bisa menjalankan `apt update` untuk security patches OS dan Docker.
6	Tidak bisa monitoring external	**MEDIUM**	Uptime monitoring services dan alerting (email, Slack) tidak bisa diterapkan.

Tabel 6.1 — Dampak Operasional Ketiadaan Internet

6.3 Kebutuhan Internet untuk CI/CD Pipeline

CI/CD (Continuous Integration / Continuous Deployment) adalah praktik standar industri untuk deployment otomatis. Dengan CI/CD, setiap perubahan kode yang di-push ke repository secara otomatis di-deploy ke server tanpa intervensi manual.

Workflow CI/CD Ideal

```
┌──────────┐   git push   ┌──────────┐  SSH deploy
┌──────────┐
│ Developer │──────────▶│  GitHub   │──────────▶│
Server    │
│          │              │  Actions  │   auto      │ 172.19.
│
│          │              │  (CI/CD)  │             │ 154.93
│
└──────────┘              └──────────┘
└──────────┘
     │                         │                         │
     │                         │ ✅ Run tests            │
     │                         │ ✅ Build validation     │
     │                         │ ✅ Auto deploy          │
     │                         │ ✅ Health check         │
     │                         │ ✅ Rollback on failure  │
     ▼                         ▼                         ▼
   1 menit                  Otomatis                  Zero
downtime
```

Perbandingan Waktu Deployment

Langkah	Tanpa Internet (Saat Ini)	Dengan Internet (CI/CD)
1. Developer commit kode	Manual: git commit + push	git commit + push
2. Transfer kode ke server	Manual: jalankan `./deploy.sh`	**Otomatis**: GitHub Actions
3. Testing	Tidak ada (manual)	**Otomatis**: run test suite
4. Restart server	Termasuk di deploy.sh	**Otomatis**: restart + health check
5. Rollback jika gagal	Manual: rsync versi lama	**Otomatis**: rollback ke commit sebelumnya
Total Waktu	**5-10 menit (manual)**	**<2 menit (otomatis)**
Ketergantungan	Developer harus available	Berjalan otomatis 24/7

Tabel 6.2 — Perbandingan Proses Deployment

6.4 Kebutuhan Internet untuk Package Management

Aplikasi CMS bergantung pada 247 npm packages yang dikelola oleh Bun package manager. Setiap package dapat memiliki security vulnerability yang ditemukan setelah deployment. Tanpa akses internet, proses update dan patching library tidak dapat dilakukan langsung pada server.

Skenario Risiko

Jika ditemukan critical vulnerability (contoh: Remote Code Execution) pada salah satu dari 247 dependencies, proses patching tanpa internet memerlukan:

1. Developer update package di PC lokal.
2. Rebuild seluruh Docker image di PC lokal.
3. Transfer Docker image (1.17 GB) ke server melalui `scp`.

4. Load image dan restart container di server.

Proses ini memakan waktu **30-60 menit** dibandingkan **5 menit** jika server memiliki akses internet (cukup `bun install` + restart).

6.5 Kebutuhan Internet untuk Security Updates

Server menjalankan Ubuntu 24.04 LTS yang secara rutin menerima security patches dari Canonical. Tanpa akses internet, perintah `apt update && apt upgrade` tidak dapat dijalankan, sehingga server rentan terhadap vulnerability yang telah diketahui.

Komponen	Current Version	Update Source	Status Tanpa Internet
Ubuntu OS	24.04.3 LTS	archive.ubuntu.com	**Tidak bisa update**
Docker Engine	29.2.1	download.docker.com	**Tidak bisa update**
Bun Runtime	1.1.20	bun.sh	**Tidak bisa update**
npm Packages (247)	Various	registry.npmjs.org	**Tidak bisa update**
Prisma ORM	5.11.0	registry.npmjs.org	**Tidak bisa update**
OpenSSL	OS bundled	archive.ubuntu.com	**Tidak bisa update**

Tabel 6.3 — Komponen yang Memerlukan Security Updates

6.6 Kebutuhan Internet untuk Docker Registry

Docker image perlu diperbarui secara berkala — baik untuk update base image (security patches) maupun untuk perubahan dependencies. Tanpa akses ke Docker Hub (`hub.docker.com`) dan npm registry (`registry.npmjs.org`), proses rebuild Docker image tidak dapat dilakukan di server.

Proses Rebuild Saat Ini (Tanpa Internet)

1. Build Docker image di PC developer lokal.

2. `docker save` image ke file tar (~1.2 GB).
3. `scp` file tar ke server (tergantung bandwidth internal).
4. `docker load` image di server.
5. Restart container dengan image baru.

Proses Rebuild Ideal (Dengan Internet)

1. SSH ke server.
2. `docker compose build && docker compose up -d` (otomatis download semua yang dibutuhkan).

6.7 Kebutuhan Internet untuk Monitoring dan Alerting

Sistem produksi memerlukan monitoring untuk mendeteksi masalah secara proaktif. Tanpa akses internet, notifikasi error tidak dapat dikirimkan melalui email, Slack, atau layanan monitoring seperti UptimeRobot, Datadog, atau Sentry.

Saat ini, jika server mengalami masalah di luar jam kerja, tidak ada mekanisme notifikasi otomatis. Masalah baru diketahui ketika ada laporan dari pengguna.

6.8 Perbandingan Workflow: Tanpa Internet vs Dengan Internet

Aspek	Tanpa Internet (Saat Ini)	Dengan Internet (Rekomendasi)
Deployment	Manual rsync dari PC developer. Bergantung pada 1 orang. 5-10 menit per deploy.	Otomatis via CI/CD. Push ke GitHub = auto deploy. <2 menit.
Rollback	Manual rsync versi lama. Rawan human error.	`git revert` + auto deploy. Cepat dan aman.
Security Patches	Tidak bisa dilakukan langsung. Harus transfer manual dari luar.	`apt update && apt upgrade`. Bisa otomatis via unattended-upgrades.
Dependency Updates	Rebuild image di luar, transfer 1.2 GB file. 30-60 menit.	`bun install` + restart. 5 menit.
Docker Rebuild	Harus build + save + scp + load. Sangat lambat.	`docker compose build`. Otomatis download.
Monitoring	Tidak ada notifikasi otomatis. Masalah diketahui dari user.	UptimeRobot / Datadog. Alert via email/Slack.
Version Control	Hanya di PC developer. Server tidak punya akses ke repository.	Git repo di server. `git pull` langsung.
Response Time (Hotfix)	30+ menit (developer harus available + SSH access).	<5 menit (push fix → auto deploy).
Single Point of Failure	**Ya** — bergantung pada 1 PC developer.	**Tidak** — siapapun dengan akses GitHub bisa deploy.

Tabel 6.4 — Perbandingan Komprehensif

6.9 Rekomendasi Konfigurasi Akses Internet

Untuk menjaga keamanan jaringan internal TPS sambil memenuhi kebutuhan operasional server, direkomendasikan konfigurasi akses internet yang terbatas (*whitelisted*) — hanya mengizinkan koneksi ke layanan-layanan yang dibutuhkan.

Opsi A: Whitelist Domain (Recommended)

Mengizinkan server mengakses hanya domain-domain tertentu melalui firewall atau proxy:

Domain	Port	Kebutuhan	Prioritas
`github.com`	443 (HTTPS)	Git repository, CI/CD	**Critical**
`registry.npmjs.org`	443	npm/Bun package download	**Critical**
`hub.docker.com`	443	Docker base images	**Critical**
`archive.ubuntu.com`	80/443	OS security updates	**Critical**
`security.ubuntu.com`	80/443	OS security patches	**Critical**
`download.docker.com`	443	Docker engine updates	**High**
`bun.sh`	443	Bun runtime updates	**High**

Tabel 6.5 — Daftar Domain yang Dibutuhkan

Opsi B: VPN atau Proxy Terbatas

Menggunakan forward proxy (seperti Squid) yang hanya mengizinkan koneksi ke domain yang di-whitelist. Semua traffic internet dari server melalui proxy, sehingga IT Security TPS dapat memonitor dan mengontrol traffic.

Opsi C: Scheduled Internet Access

Memberikan akses internet hanya pada waktu tertentu (contoh: jam maintenance 00:00-06:00) untuk keperluan update dan deployment, dan menutup akses di luar jam tersebut.

Rekomendasi: Opsi A (Whitelist Domain) memberikan keseimbangan terbaik antara keamanan dan operasional. Server hanya bisa mengakses layanan yang benar-benar dibutuhkan, sementara IT Security tetap memiliki kontrol penuh.

BAB VII

PENGUJIAN DAN VALIDASI

7.1 Pengujian Fungsional

Halaman	URL	HTTP	Size	Status
Homepage	`/`	200	237 KB	✅ Pass
Unduh Dokumen	`/unduh-dokumen`	200	20 KB	✅ Pass
Berita	`/berita/press-release`	200	64 KB	✅ Pass
Admin Dashboard	`/backend/tpsadmin/dashboard`	200	OK	✅ Pass
Admin Edit	`/backend/tpsadmin/edit/:id`	200	OK	✅ Pass
File Download	`/_file/media/...`	200	6.7 MB	✅ Pass
File Upload	`/_upload`	200	OK	✅ Pass
File Redirect (legacy)	`/file/media/...`	301→200	OK	✅ Pass
Karir	`/karir`	200	OK	✅ Pass

Tabel 7.1 — Hasil Pengujian Fungsional Website

7.2 Pengujian Deployment

Test Case	Expected	Actual	Status
`./deploy.sh` (full deploy)	Sync + restart + healthy	4 dirs synced, container Up (healthy), HTTP 200	✅ Pass
`./deploy.sh --restart`	Restart only	Container restarted, Up (healthy)	✅ Pass
`./deploy.sh --sync-only`	Sync + clear cache	4 dirs synced, cache cleared	✅ Pass
Volume mount verification	6 mounts active	6 bind mounts confirmed	✅ Pass
Health check after deploy	Healthy within 40s	Healthy within 25s	✅ Pass

Tabel 7.2 — Hasil Pengujian Deploy Script

BAB VIII
KESIMPULAN DAN REKOMENDASI

8.1 Kesimpulan

1. **Application server** (172.19.154.93) telah berhasil diinstall dan dikonfigurasi dengan Docker Engine, menjalankan aplikasi CMS pada container `tps-app` yang terhubung ke database server (172.19.154.92).
2. **Empat kendala teknis** telah berhasil ditemukan dan diperbaiki: Prisma initialization failure, file upload bugs (3 issues), file URL routing, dan tilde encoding pada URL legacy.
3. **Sistem deployment** menggunakan volume mount + rsync telah di-setup dan divalidasi berfungsi dengan baik melalui script `deploy.sh`.
4. **Ketiadaan akses internet** pada server merupakan kendala operasional yang signifikan, berdampak pada kecepatan deployment, keamanan (tidak bisa patch), dan menciptakan single point of failure pada proses deployment.
5. Seluruh halaman website dan fitur admin (termasuk upload file) telah diverifikasi berfungsi normal.

8.2 Rekomendasi

No	Rekomendasi	Prioritas	Justifikasi
1	**Memberikan akses internet terbatas (whitelisted) pada server**	CRITICAL	Diperlukan untuk CI/CD, security patches, dependency updates, dan Docker rebuild. Tanpa ini, server bergantung pada deployment manual dan rentan terhadap vulnerability yang tidak bisa di-patch.
2	Setup CI/CD pipeline (GitHub Actions)	HIGH	Menghilangkan single point of failure dan mempercepat deployment dari 10 menit manual menjadi <2 menit otomatis. Memerlukan internet pada server.
3	Automated backup database	HIGH	Jadwal backup harian menggunakan `pg_dump` dan penyimpanan di lokasi terpisah untuk disaster recovery.
4	Monitoring dan alerting	HIGH	Implementasi monitoring uptime dan error alerting agar masalah terdeteksi proaktif, bukan dari laporan pengguna. Memerlukan internet untuk notifikasi.
5	SSL/TLS certificate	HIGH	Mengamankan koneksi HTTP dengan HTTPS menggunakan Let's Encrypt atau certificate internal. Memerlukan internet untuk Let's Encrypt.
6	Upgrade Prisma ORM	MEDIUM	Versi 5.11.0 memiliki bug schema parser. Upgrade ke versi terbaru diperlukan. Memerlukan internet untuk download package.
7	Scheduled security updates	MEDIUM	Jadwal patch OS dan Docker secara berkala (minimal bulanan). Memerlukan internet.

Tabel 8.1 — Daftar Rekomendasi

CATATAN PENTING: Dari 7 rekomendasi di atas, **5 di antaranya memerlukan akses internet pada server**. Tanpa akses internet, hanya rekomendasi #3 (backup database) dan sebagian #6 (dengan workaround manual) yang dapat direalisasikan. Hal ini menekankan urgensitas pemberian akses internet terbatas pada server.

LAMPIRAN

Lampiran A — Dockerfile Lengkap

```
FROM oven/bun:1.1.20-debian

RUN apt-get update && apt-get install -y \
    curl git openssl \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY . ./
RUN bun install --production
RUN mkdir -p /app/uploads /app/logs
RUN cd app/db && rm -f .env && \
    bunx prisma generate --schema=prisma/schema.prisma

ENV NODE_ENV=production
ENV PORT=3300
EXPOSE 3300

HEALTHCHECK --interval=30s --timeout=10s \
    --start-period=40s --retries=3 \
    CMD curl -f http://localhost:3300/ || exit 1

CMD ["bun", "run", "prod"]
```

Lampiran B — Docker Compose Lengkap

```
services:
  tps-app:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: tps-app
    restart: unless-stopped
    ports:
      - "3300:3300"
    environment:
      - NODE_ENV=production
      - DATABASE_URL=postgresql://postgres:***@172.19.154.92:5432/
tps_web
      - PORT=3300
    volumes:
      - /opt/tps-app/pkgs:/app/pkgs
      - /opt/tps-app/app/srv:/app/app/srv
      - /opt/tps-app/app/db:/app/app/db
      - /opt/tps-app/app/web/deploy:/app/app/web/deploy
      - /opt/tps-data/uploads:/data/files
      - /opt/tps-data/logs:/app/logs
    networks:
      - tps-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3300/"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s

networks:
  tps-network:
    driver: bridge
```

Lampiran C — Deploy Script (deploy.sh)

```
#!/bin/bash
set -e

SERVER="root@172.19.154.93"
REMOTE_APP="/opt/tps-app"
LOCAL_SRC="tps-core/tps"

# Sync source code
rsync -az --delete --exclude='node_modules' \
    "$LOCAL_SRC/pkgs/" "$SERVER:$REMOTE_APP/pkgs/"
rsync -az --delete \
    "$LOCAL_SRC/app/srv/" "$SERVER:$REMOTE_APP/app/srv/"
rsync -az --exclude='node_modules' --exclude='.env' \
    "$LOCAL_SRC/app/db/" "$SERVER:$REMOTE_APP/app/db/"
rsync -az \
    "$LOCAL_SRC/app/web/deploy/" "$SERVER:$REMOTE_APP/app/web/
deploy/"

# Restart + health check
ssh $SERVER "docker restart tps-app"
sleep 12
ssh $SERVER "docker ps --filter name=tps-app --format '{{.Status}}'"
ssh $SERVER "curl -s -o /dev/null -w '%{http_code}' http://
localhost:3300/"
```

Lampiran D — Daftar File yang Diperbaiki

No	File	Perubahan
1	`pkgs/utils/prisma.ts`	Pisahkan prisma generate dan PrismaClient import ke try-catch terpisah
2	`app/srv/api/_tpsadmin_edit.ts`	Fix FormData body, hapus location.reload(), update preview di DOM
3	`pkgs/api/_upload.ts`	Fix directory creation dengan absolute path
4	`pkgs/server/create.ts`	Tambah redirect /file/* → /_file/*, konversi ~ ke %20
5	`deploy/docker-compose.yml`	Tambah volume mounts untuk source code directories
6	`deploy.sh`	Update workflow rsync + restart (tanpa docker cp)

Dokumen ini bersifat **RAHASIA** dan hanya untuk penggunaan internal PT Terminal Petikemas Surabaya dan PT PRIMALINK.